



MECÁNICA CUÁNTICA

Proyecto de Introducción a la Computación Cuántica

Autora: Sandra Martínez Castellanos

Profesor: Felipe José Llanes Estrada

Fecha de inicio: 22 de Abril de 2026

Fecha de entrega: 30 de Abril de 2026

ÍNDICE

I. Práctica 1: Introducción a la programación en QUIRK	2
1. Puertas cuánticas de 1 qubit.	2
1.1. Productos de puertas cuánticas.	2
1.2. Circuitos	3
2. Transformaciones unitarias	5
2.1. Transformaciones unitarias o no	5
2.2. Verificar en QUIRK que σ_x , σ_y and σ_z , son unitarias	5
3. Puertas cuánticas de varios qubits	6
3.1. Puertas de control: CNOT y otras	6
3.2. Construcción de estados de Bell entrelazados	7
3.3. Imposibilidad de clonar qubits, incluso mediante la puerta CNOT	8
3.4. Entrelazamiento cuántico de qubits distantes mediante puertas SWAP	10
II. Práctica 2: Introducción a la programación en QISKIT	12
1. ¿Por qué el histograma tiene una única barra correspondiente a $ 1\rangle$?	12
2. Escribir el estado inicial de la línea de código	12
3. Ejecute el circuito varias veces y anote los resultados obtenidos en binario	12
4. Escribe un bucle que genere 32 números aleatorios y dibuja el histograma	12
5. Escribir todos los métodos que se han aprendido	13
6. Comprobar que la función siguiente implementa la transformación R_k	13
7. Escribir el estado resultante de aplicar la transformación al estado inicial $ 010\rangle$	13
8. Ejercicio completo: Implementar la QFT como una puerta que actúe sobre 5 qubits	14
III. Práctica 3: Ejecución en los ordenadores de IBM	17
1. Generación de números aleatorios	17
1.1. Definir el mismo circuito de la Práctica 2	17
1.2. Comentario de los resultados	17
2. Algoritmo de DEUTSCH	17
2.1. Demostrar la unitariedad del siguiente operador	17
2.2. Completar los códigos	18
2.3. Medir en f	19
2.4. Medir en g	19
2.5. Demostración si $f(0) \neq f(1)$	20
2.6. Definir un circuito para implementar el algoritmo de Deutsch con f y g	20
2.7. Comparación con el resultado de antes	22
3. Desigualdades de Bell CHSH	22
3.1. Definir el circuito	22
3.2. Definir los cuatro productos de observables	22
3.3. Calcular el valor de la desigualdad CHSH	22
3.4. Valor experimental para CHSH	22
IV. Anexos	23

Resumen

Este proyecto educacional de Computación Cuántica Práctica pretende ser una introducción al manejo de los programas QUIRK y QISKIT mediante tres sesiones de prácticas.

I. PRÁCTICA 1: INTRODUCCIÓN A LA PROGRAMACIÓN EN QUIRK

1. Puertas cuánticas de 1 qubit.

1.1. Productos de puertas cuánticas.

a) $H \cdot H = \mathbb{I}$

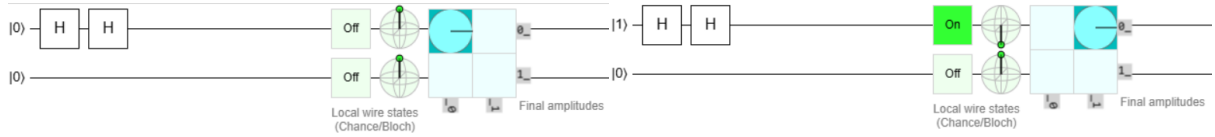


Figura 1: Salida del circuito $H \cdot H$ para $|0\rangle$ y $|1\rangle$.

b) $X \cdot X = \mathbb{I}$

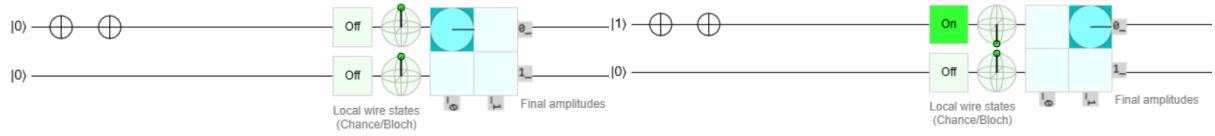


Figura 2: Salida del circuito $X \cdot X$ para $|0\rangle$ y $|1\rangle$.

c) $Y \cdot Y = \mathbb{I}$

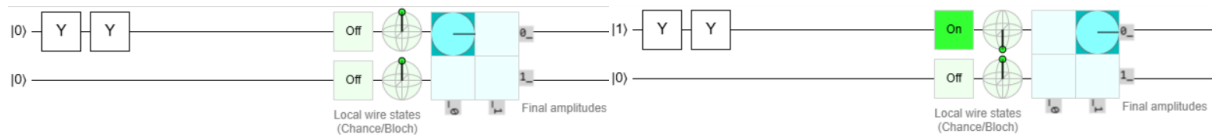


Figura 3: Salida del circuito $Y \cdot Y$ para $|0\rangle$ y $|1\rangle$.

d) $Z \cdot Z = \mathbb{I}$

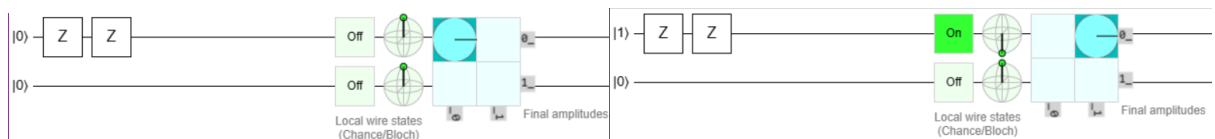


Figura 4: Salida del circuito $Z \cdot Z$ para $|0\rangle$ y $|1\rangle$.

e) $X^{1/2} \cdot X^{1/2} = X$

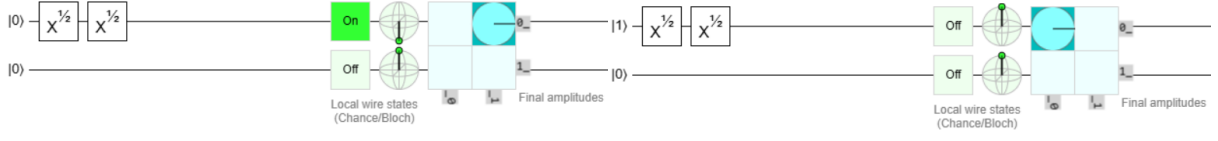


Figura 5: Salida del circuito $X^{1/2} \cdot X^{1/2}$ para $|0\rangle$ y $|1\rangle$.

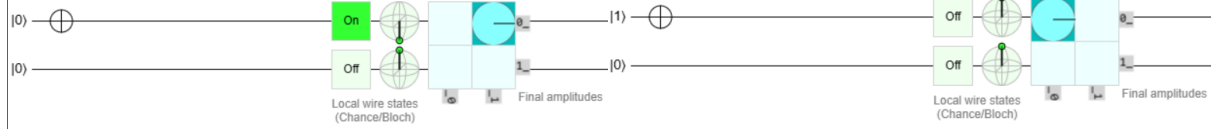


Figura 6: Equivalente al circuito anterior.

f) $X^{1/4} \cdot X^{1/4} = X^{1/2}$

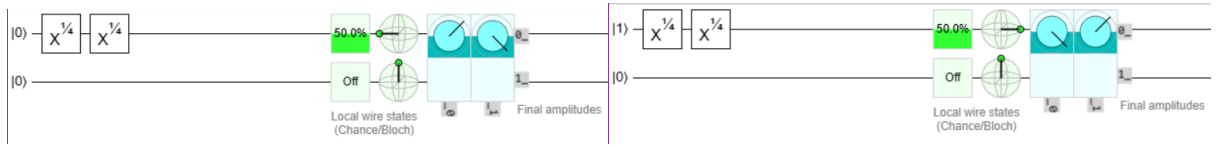


Figura 7: Salida del circuito $X^{1/4} \cdot X^{1/4}$ para $|0\rangle$ y $|1\rangle$.

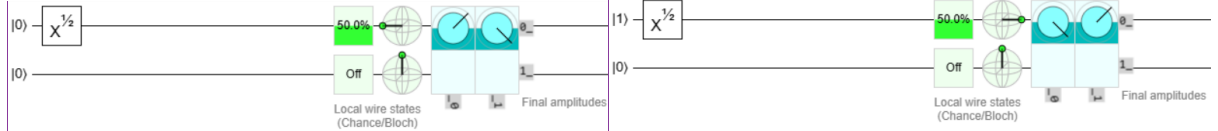


Figura 8: Equivalente al circuito anterior.

g) $H \cdot Y \cdot H = -Y$, introduciendo una fase global $e^{i\delta} = -1$ respecto del operador Y .

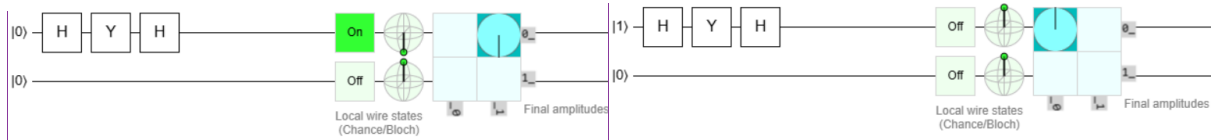


Figura 9: Salida del circuito $H \cdot Y \cdot H$ para $|0\rangle$ y $|1\rangle$.

1.2. Circuitos

a) $|1\rangle$

$X|0\rangle = |1\rangle$

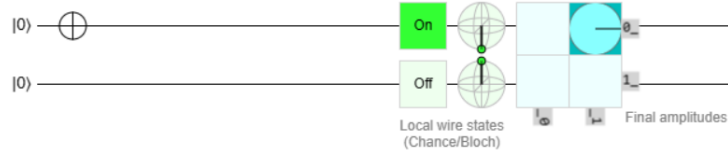


Figura 10: Salida del circuito a).

b) $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$
 $H|0\rangle = |+\rangle$

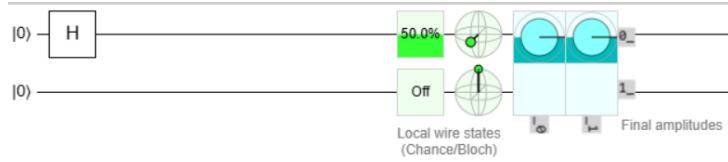


Figura 11: Salida del circuito b).

c) $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$
 $X \cdot H|0\rangle = |-\rangle$

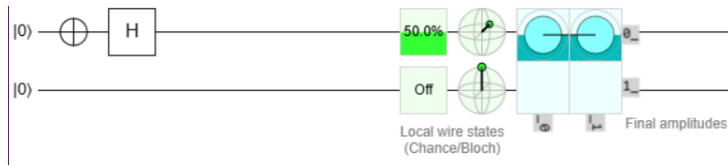


Figura 12: Salida del circuito c).

d) $|R\rangle = \frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle)$
 $H \cdot S|0\rangle = |R\rangle$

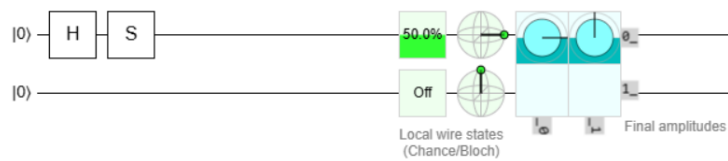


Figura 13: Salida del circuito d).

e) $|L\rangle = \frac{1}{\sqrt{2}}(|0\rangle - i|1\rangle)$
 $H \cdot S \cdot X|0\rangle = |L\rangle$

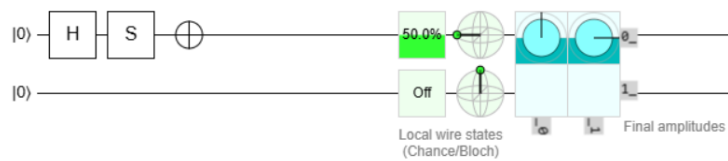


Figura 14: Salida del circuito e).

2. Transformaciones unitarias

Preservan el producto escalar complejo: $U^\dagger U = \mathbb{I}$

U es siempre invertible: $U^{-1} = U^\dagger$

2.1. Transformaciones unitarias o no

1. Ordenar alfabéticamente: NO es unitaria. Si tenemos un estado $|ba\rangle^1$ y lo ordeno alfabéticamente como $|ab\rangle$. Tomando el producto escalar: $(\langle a|\langle b|)(|b\rangle|a\rangle) = 0$ y transformado: $(\langle a|\langle b|)(|a\rangle|a\rangle) = 1$.
2. Aplicar una permutación: SÍ es unitaria. Es invertible, reordeno los subsistemas. Además, el producto escalar no cambia ya que se hace subsistema a subsistema.
3. Sumar dos elementos módulo d y no guardar ninguno: NO es unitaria. No es invertible: tomando el estado base $|a\rangle|b\rangle$, después de la transformación queda $|a\oplus b\rangle$, borrando el segundo. Si se pone de entrada el estado $|a+1\rangle|b-1\rangle$, también se llega al mismo estado final. Lo que demuestra la no invertibilidad.
4. Sumar dos elementos módulo d y guardar uno: SÍ es unitaria. Teniendo uno de los elementos guardado, se puede recuperar el otro como $a \equiv (a \oplus b) \ominus b$, ya que al estar acotados entre 0 y $d-1$, la operación es invertible. Además, se preserva el producto escalar: $\langle ab|a'b'\rangle = \langle (a\oplus b)b|(a'\oplus b')b'\rangle^2$:

$$\begin{aligned}\langle ab|a'b'\rangle &= \langle a|a'\rangle \langle b|b'\rangle \\ \langle (a\oplus b)b|(a'\oplus b')b'\rangle &= \langle a\oplus b|a'\oplus b'\rangle \langle b|b'\rangle\end{aligned}\tag{1}$$

- Si $(b \neq b')$: $\langle b|b'\rangle = 0$ y los dos productos escalares de arriba son 0 .
- Si $(b = b')$: $\langle b|b'\rangle = 1$ y se puede simplificar a: $\langle a\oplus b|a'\oplus b'\rangle$. Y tenemos dos subespacios para a :
 - Si $(a \neq a')$: Entonces $\langle a|a'\rangle = 0$ y $\langle a\oplus b|a'\oplus b'\rangle = 0$
 - Si $(a = a')$: Entonces $\langle a|a\rangle = 1$ y $\langle a\oplus b|a\oplus b\rangle = 1$

Se concluye entonces que el producto escalar se preserva y que U es unitario: $U^\dagger U = \mathbb{I}$.

5. Simetrizar dos estados: SÍ es unitaria.

Como la suma de las probabilidades debe ser 1 , el factor $\frac{1}{\sqrt{2}}$ lo asegura. Con la antisimetrización ocurre lo mismo.

6. Filtrar una lista: NO es unitaria.

Como de una lista se está borrando parte de la información (ejemplo: todos los negativos), la operación no es invertible. Demostración: $(\langle 1|\langle -1|)(|1\rangle|-2\rangle) = 0$, pero si solo nos quedamos con los positivos se queda en $\langle 1|1\rangle = 1$.

2.2. Verificar en QUIRK que σ_x , σ_y and σ_z , son unitarias

Cada una de las puertas de Pauli (X, Y, Z), han sido vistas en el ejercicio 1.1 b), c), d). Se ha visto que $M \cdot M = \mathbb{I}$, por lo que sabemos que $M^{-1} = M$. Son unitarias además porque se cumple que $M = M^\dagger$ al saber que las matrices de Pauli son autoadjuntas.

¹ $|ba\rangle = |b\rangle|a\rangle$

² $\langle (a\oplus b)b|(a'\oplus b')b'\rangle = \langle (a \otimes b|\langle b|)(|a'\otimes b')|b'\rangle$

3. Puertas cuánticas de varios qubits

3.1. Puertas de control: CNOT y otras

Siguiendo los pasos:

- 2. Con la puerta NOT el segundo qubit pasa del estado $|0\rangle$ a $|1\rangle$:

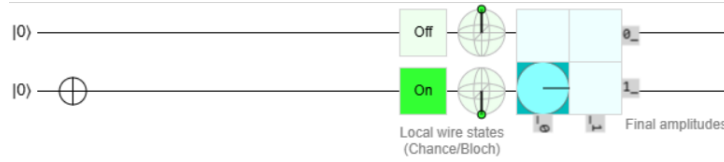


Figura 15: Salida del circuito siguiendo el punto 2.

- 3. Con este cambio el primer qubit no cambia y el segundo vuelve al estado $|0\rangle$:

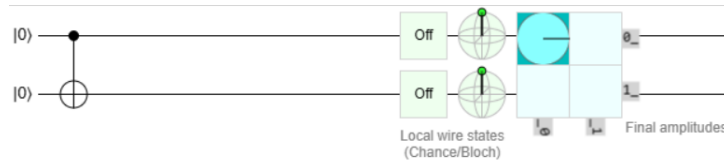


Figura 16: Salida del circuito siguiendo el punto 3.

- 4. El primer qubit no cambia y le indica a la puerta que debe cambiar el segundo qubit de $|0\rangle$ a $|1\rangle$:

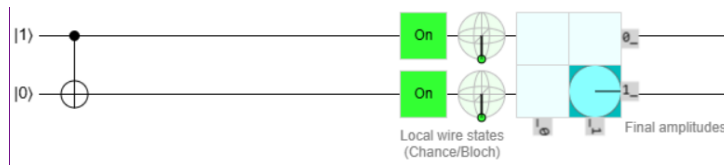


Figura 17: Salida del circuito siguiendo el punto 4.

- 5. Repitiendo con un anticontrol:
 - (3) La instrucción es contraria a la puerta de control: el primer qubit no cambia e indica a la puerta que el segundo debe pasar de $|0\rangle$ a $|1\rangle$:

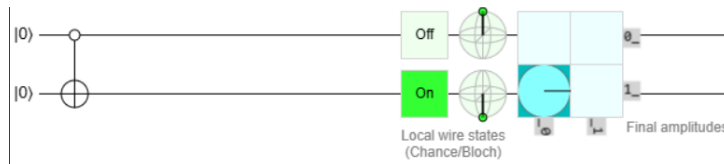


Figura 18: Salida del circuito siguiendo el punto 5 (3).

- (4) Ninguno de los dos qubits cambia:

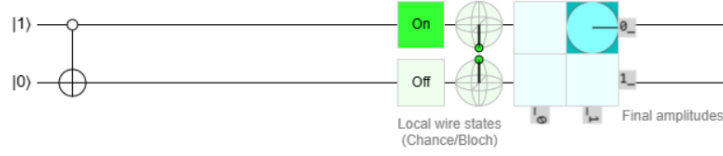


Figura 19: Salida del circuito siguiendo el punto 5 (4).

- 6. Repitiendo con una puerta de Hadamard en lugar de con un NOT:
 - (2) El primer qubit se queda igual y el segundo pasa de $|0\rangle$ a $|+\rangle$:

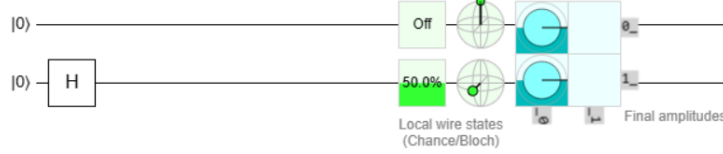


Figura 20: Salida del circuito siguiendo el punto 6 (2).

- (3) Ninguno de los dos qubits cambia:

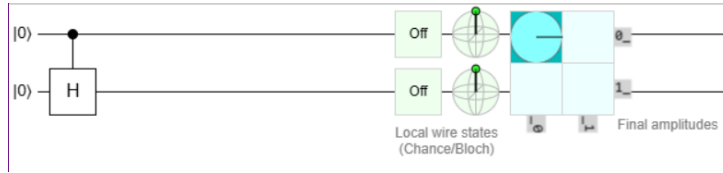


Figura 21: Salida del circuito siguiendo el punto 6 (3).

- (4) El primero no cambia e indica que el segundo debe pasar del estado $|0\rangle$ a $|+\rangle$:

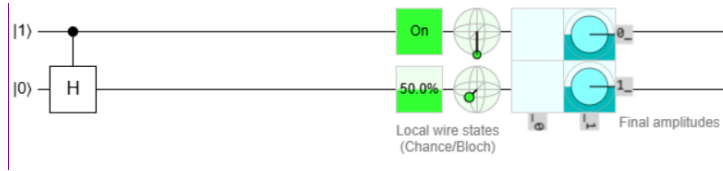


Figura 22: Salida del circuito siguiendo el punto 6 (4).

El procedimiento para el *anticontrol* con la puerta de Hadamard es análogo al seguido con la puerta NOT.

3.2. Construcción de estados de Bell entrelazados

1. $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|0\rangle \otimes |0\rangle + |1\rangle \otimes |1\rangle)$

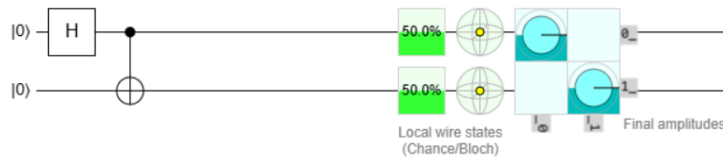


Figura 23: Salida del circuito que construye el estado $|\Phi^+\rangle$ de Bell.

2. $|\Phi^-\rangle = \frac{1}{\sqrt{2}}(|0\rangle \otimes |0\rangle - |1\rangle \otimes |1\rangle)$

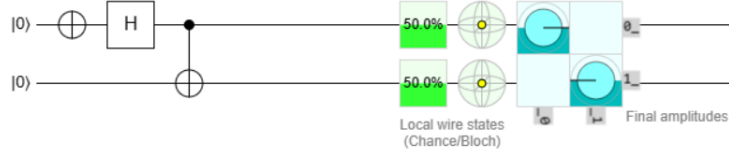


Figura 24: Salida del circuito que construye el estado $|\Phi^-\rangle$ de Bell.

3. $|\Psi^+\rangle = \frac{1}{\sqrt{2}}(|0\rangle \otimes |1\rangle + |1\rangle \otimes |0\rangle)$

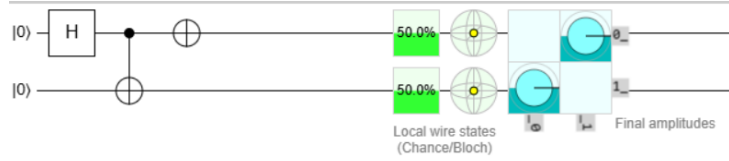


Figura 25: Salida del circuito que construye el estado $|\Psi^+\rangle$ de Bell.

Se podría haber hecho también con *anticontrol*.

4. $|\Psi^-\rangle = \frac{1}{\sqrt{2}}(|0\rangle \otimes |1\rangle - |1\rangle \otimes |0\rangle)$

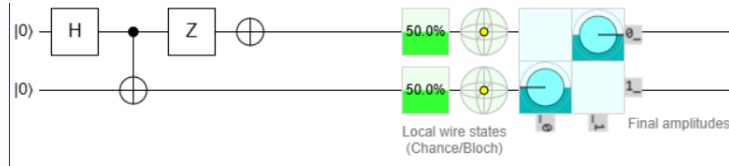


Figura 26: Salida del circuito que construye el estado $|\Psi^-\rangle$ de Bell.

3.3. Imposibilidad de clonar qubits, incluso mediante la puerta CNOT

1. Partiendo de $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$:

(a) Estado de 2 qubits si cada qubit ha sido inicializado a $|-\rangle$:

$$\begin{aligned} |\Psi\rangle &= |-\rangle \otimes |-\rangle = \frac{1}{2}(|0\rangle - |1\rangle) \otimes (|0\rangle - |1\rangle) \\ &= \frac{1}{2}(|00\rangle - |01\rangle - |10\rangle + |11\rangle) \end{aligned}$$

Se puede igualar al vector para verlo de forma ilustrativa con QUIRK:

$$|-\rangle \otimes |-\rangle = \frac{1}{2}(1, -1, -1, 1)^T$$

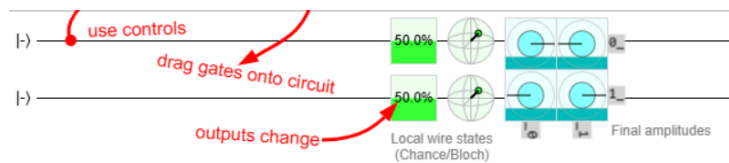


Figura 27: Estado $|-\rangle \otimes |-\rangle$ en Quirk.

(b) Estado que resulta de aplicar un CNOT al primer qubit inicializado en $|-\rangle$ e inicializando el segundo en $|0\rangle$:

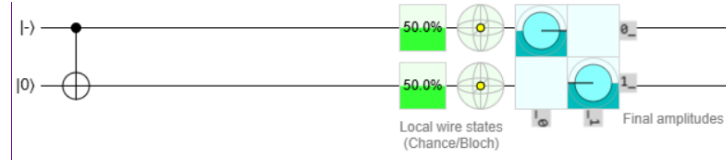


Figura 28: Circuito con CNOT.

El resultado es:

$$|\Phi^-\rangle = \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle) = \frac{1}{\sqrt{2}}(1, 0, 0, -1)^T$$

(c) No se puede clonar el estado del primer qubit en el segundo, los apartado (a) y (b) dan soluciones distintas. Es cierto que podría parecer que se ha clonado el estado del primer qubit en el segundo ya que en el apartado (b) el estado sale sin coherencias.

2. Utilizando dos puertas de Hadamard se preparan dos registros cuánticos:

- A: $|-\rangle \otimes |-\rangle$
- B: Primer qubit $|-\rangle$. Segundo qubit aplicando un CNOT a $|0\rangle$

(a) Probabilidad de medir $|1\rangle$ en cada uno de los 4 qubits:

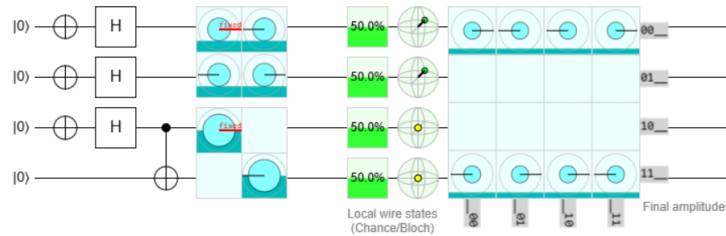


Figura 29: Circuito con puertas H .

En cada uno de ellos la probabilidad es de $1/2$.

(b) Diferencias entre las cuatro esferas de Bloch:

Las dos primeras son estados puros ya que se encuentran en la superficie de la esfera y las dos últimas son estados mezcla ya que están en el interior (se encuentran en $(0.0.0)$).

(c) Aplicando una puerta de Hadamard adicional:



Figura 30: Circuito con puertas H y una adicional.

En el registro A, como el estado $|-\rangle$ se crea como $H|1\rangle$ o $HX|0\rangle$, al aplicar antes de medir una puerta H, vuelve al estado $|1\rangle$, porque H es su propia inversa. Para el registro B, el tercer y cuarto qubit se han entrelazado por el CNOT, al aplicar H, no se devuelve $|1\rangle$ como en el registro A, si no que sale un estado entrelazado. Como se puede ver, el CONTROL hace efecto sobre el estado del qubit en cuestión.

(d) Efecto de $H \otimes H$:

$$H \otimes H = \frac{1}{2} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix} \quad (2)$$

El estado que nos dan es $|\Phi^-\rangle = \frac{1}{\sqrt{2}}(1, 0, 0, -1)^T$, entonces:

$$H \otimes H |\Phi^-\rangle = \frac{1}{2\sqrt{2}} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \\ -1 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix} \quad (3)$$

Que explica lo obtenido en QUIRK.

3.4. Entrelazamiento cuántico de qubits distantes mediante puertas SWAP

1. Circuito de 4 cubits que genere 2 parejas de Bell: $|\Phi^+\rangle$ (1,3) y $|\Phi^-\rangle$ (2,4). Se prepara el estado:

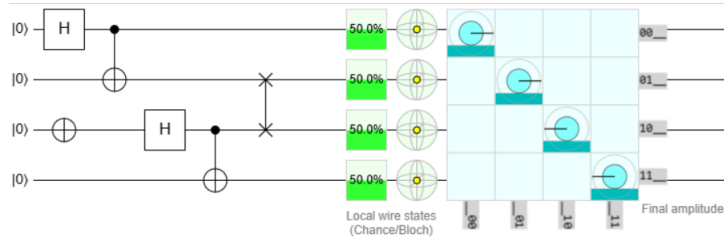


Figura 31: Circuito de 4 qubits con 2 parejas de Bell.

2. Verificar que los cubits (1,3) y (2,4) están entrelazados. Se debería recuperar el estado inicial $|0000\rangle$

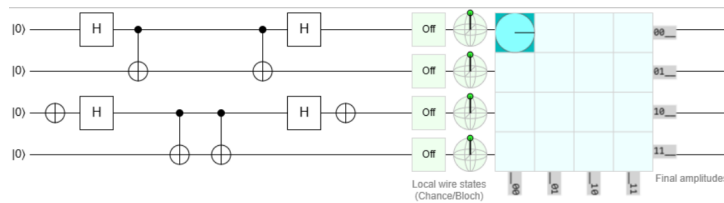


Figura 32: Circuito que verifica el entrelazamiento.

Que demuestra que el circuito anterior entrelaza correctamente los cubits (1,3) y (2,4) y además que el circuito inverso recupera el estado inicial. $(\mathbb{I} \otimes \mathbb{I} \otimes (CNOT))^2 = ((CNOT) \otimes \mathbb{I} \otimes \mathbb{I})^2 = (\mathbb{I} \otimes \mathbb{I} \otimes \mathbb{I} \otimes H)^2 = (\mathbb{I} \otimes H \otimes \mathbb{I} \otimes \mathbb{I})^2 = \mathbb{I} \otimes \mathbb{I} \otimes \mathbb{I} \otimes \mathbb{I}$.

3. Puerta que intercambia los valores de 2 cubits utilizando 3 puertas CNOT y comparación con SWAP.

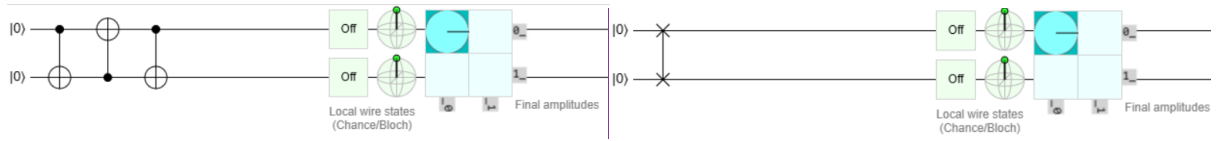


Figura 33: Circuito equivalente a CNOT con SWAP para $|00\rangle$.

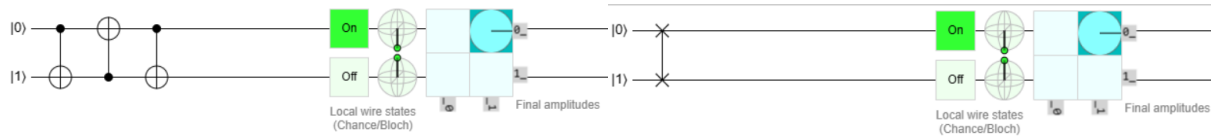


Figura 34: Circuito equivalente a CNOT con SWAP para $|01\rangle$.

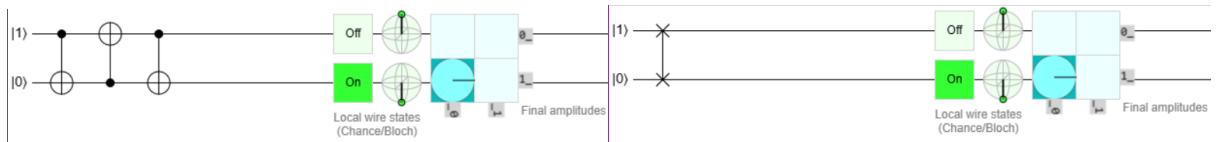


Figura 35: Circuito equivalente a CNOT con SWAP para $|10\rangle$.

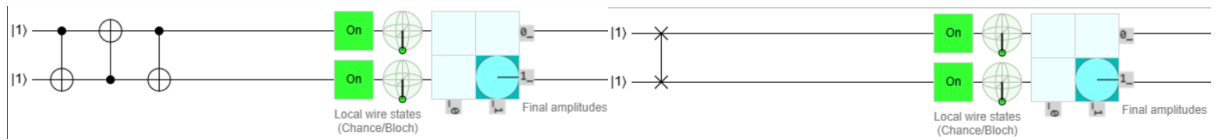


Figura 36: Circuito equivalente a CNOT con SWAP para $|11\rangle$.

4. Permutación $(1,2,3,4,5) \rightarrow (2,4,1,3,5)$: $(1,2,3,4,5) \rightarrow (2,1,3,4,5) \rightarrow (2,1,4,3,5) \rightarrow (2,4,1,3,5)$

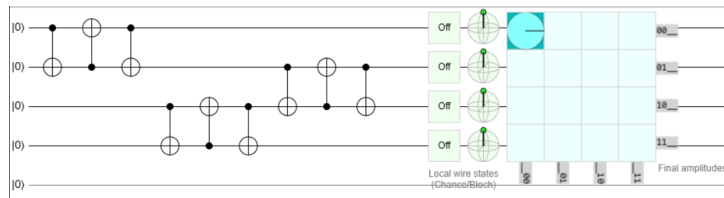


Figura 37: Circuito de permutaciones.

II. PRÁCTICA 2: INTRODUCCIÓN A LA PROGRAMACIÓN EN QISKIT

Aquí se resolverán las preguntas halladas en el cuaderno de Jupyter.

1. *¿Por qué el histograma tiene una única barra correspondiente a $|1\rangle$?*

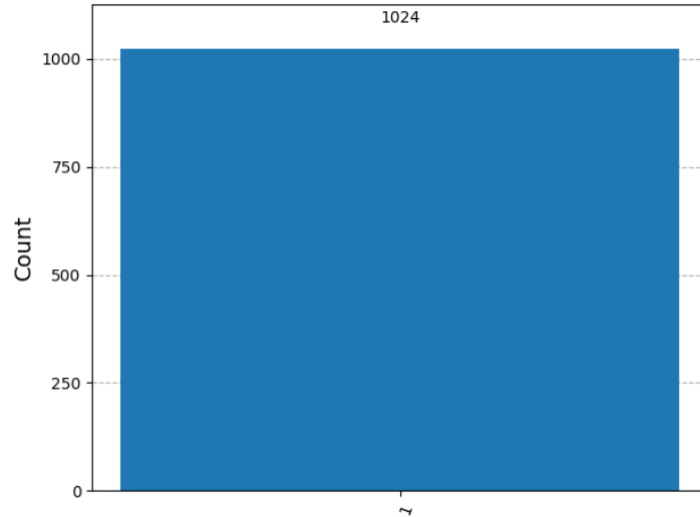


Figura 38: Histograma de $|1\rangle$.

Partiendo del estado $|1\rangle$, al medir sobre la base computacional, solamente se obtendrá la medida asociada al estado (1), da igual cuántas veces se mida, se mide este estado con probabilidad 1. La amplitud de probabilidad del estado $|0\rangle$ es nula.

2. *Escribir el estado inicial de la línea de código*

El estado en el código: `init_state = np.kron([1,1],np.kron([1,1],np.kron([1,1],[1,1])))`.

Teniendo 4 qubits con una dimensión de $2^4 = 16$ y sabiendo que la lista `[1,1]` es el estado $(|0\rangle + |1\rangle)$.

El estado que obtenemos es: $(|0\rangle + |1\rangle) \times (|0\rangle + |1\rangle) \times [(|0\rangle + |1\rangle) \times (|0\rangle + |1\rangle)] = (|0\rangle + |1\rangle) \times (|0\rangle + |1\rangle) \times [|00\rangle + |01\rangle + |10\rangle + |11\rangle]$

Con $|a\rangle \otimes |b\rangle \equiv |ab\rangle$

3. *Ejecute el circuito varias veces y anote los resultados obtenidos en binario*

Al correr el circuito $N = 10$ veces, los resultados obtenidos han sido: 0101: 1, 1010: 1, 0001: 2, 0111: 1, 1000: 1, 0000: 1, 0100: 1, 0110: 2.

4. *Escribe un bucle que genere 32 números aleatorios y dibuja el histograma*

Se ha utilizado el siguiente programa:

```
1 result = sim.run(qrng,shots=32).result()
2 rn_bin = result.get_counts(qrng)
3 rn_dec = {int(key, 2) : value for key, value in rn_bin.items()}
4 plot_histogram(rn_dec)
```

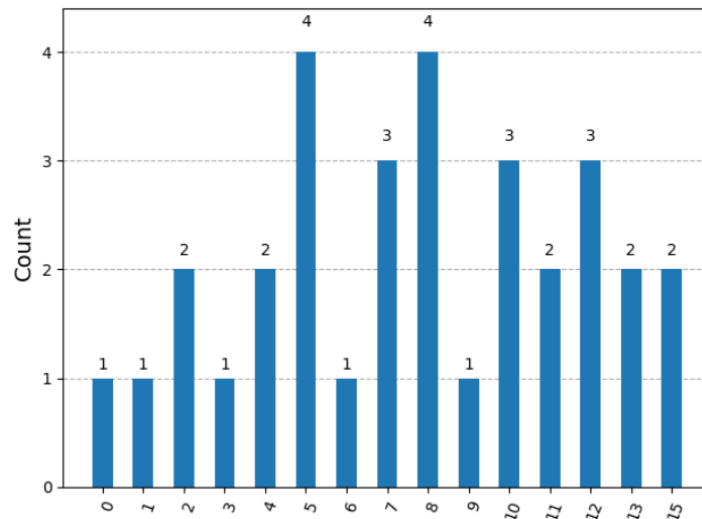


Figura 39: Histograma de los números aleatorios.

5. Escribir todos los métodos que se han aprendido

Se han generado circuitos aleatorios de qubits inicializados con amplitudes de probabilidad iguales para todos los estados. Se han almacenado las medidas en bits clásicos. Por último, se ha dibujado un histograma con los valores decimales en lugar de binarios.

6. Comprobar que la función siguiente implementa la transformación R_k

```

1 from qiskit.circuit.library import PhaseGate, XGate
2 def R(k):
3     return PhaseGate((2.*np.pi)/pow(2,k))

1 Example1 = QuantumCircuit(1)
2 Example1.append(XGate(), [0])
3 Example1.append(R(2), [0])
4 Example1.draw(output='mpl')

```



Figura 40: Circuito que comprueba la función R_k

```

1 psi4 = Statevector(Example1)
2 psi4.draw('latex')

```

Queda: $i|1\rangle$

7. Escribir el estado resultante de aplicar la transformación al estado inicial $|010\rangle$

```

1 qft_circuit = QuantumCircuit(3, name = 'QFT')
2 qft_circuit.initialize([1,0], 0)
3 qft_circuit.initialize([0,1], 1)
4 qft_circuit.initialize([1,0], 2)
5 qft_circuit.h(2)

```

```

6 qft_circuit.append(R(2).control(), [2,1])
7 qft_circuit.draw(output='mpl')

```

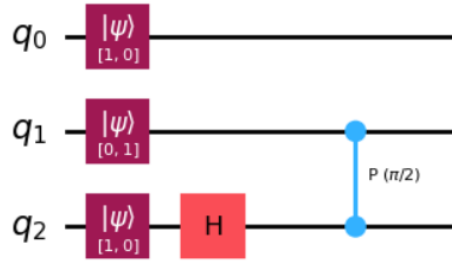


Figura 41: Salida del circuito aplicado al $|010\rangle$

```

1 phi5 = Statevector(qft_circuit)
2 phi5
3 phi5.draw('latex')

```

Queda: $\frac{\sqrt{2}}{2}|010\rangle + \frac{\sqrt{2}i}{2}|110\rangle$

8. Ejercicio completo: Implementar la QFT como una puerta que actúe sobre 5 qubits

Se aplica a los 5 estados del cuaderno de Jupyter.

```

1 qft5 = QuantumCircuit(5, name = 'QFT5')
2 # Transforming qubit q_4:
3 qft5.h(4)
4 qft5.barrier()
5 qft5.append(R(2).control(), [4,3])
6 qft5.barrier()
7 qft5.append(R(3).control(), [4,2])
8 qft5.barrier()
9 qft5.append(R(4).control(), [4,1])
10 qft5.barrier()
11 qft5.append(R(5).control(), [4,0])
12 qft5.barrier()
13 # Transforming qubit q_3:
14 qft5.h(3)
15 qft5.barrier()
16 qft5.append(R(2).control(), [3,2])
17 qft5.barrier()
18 qft5.append(R(3).control(), [3,1])
19 qft5.barrier()
20 qft5.append(R(4).control(), [3,0])
21 qft5.barrier()
22 # Transforming qubit q_2:
23 qft5.h(2)
24 qft5.barrier()
25 qft5.append(R(2).control(), [2,1])
26 qft5.barrier()
27 qft5.append(R(3).control(), [2,0])
28 qft5.barrier()
29 # Transforming qubit q_1:
30 qft5.h(1)
31 qft5.barrier()
32 qft5.append(R(2).control(), [1,0])
33 qft5.barrier()
34 # Transforming qubit q_0:
35 qft5.h(0)

```

```

36 qft5.barrier()
37 # SWAP
38 qft5.swap(0,4)
39 qft5.barrier()
40 qft5.swap(1,3)
41 qft5.draw(output='mpl')

```

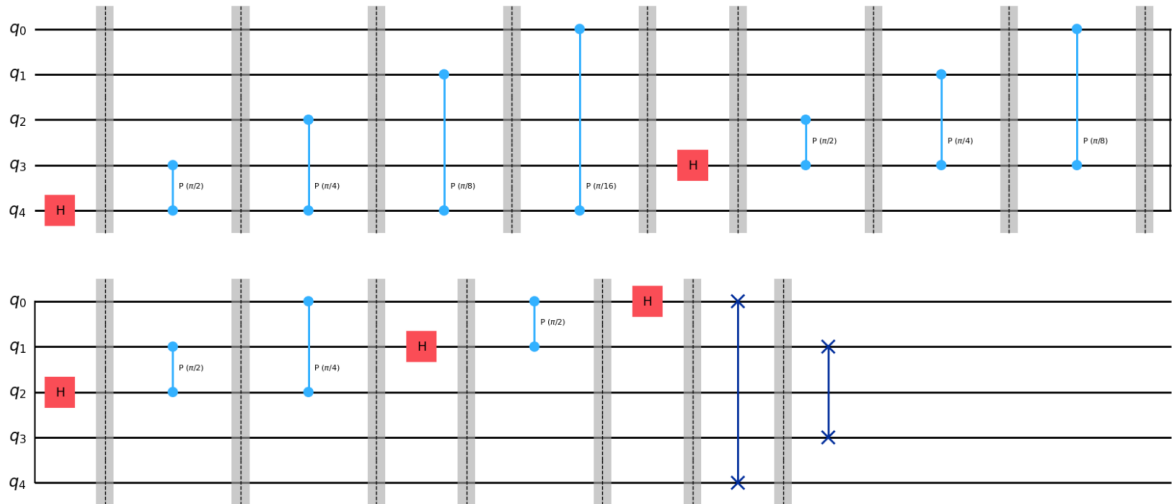


Figura 42: Circuito que actúa sobre 5 qubits (con barreras para que no se desordenen las puertas en la imagen).

a) $|\phi_1\rangle = \frac{1}{2^{5/2}}(|0\rangle + |1\rangle)(|0\rangle + |1\rangle)(|0\rangle + |1\rangle)(|0\rangle + |1\rangle)(|0\rangle - |1\rangle)$

```

1 qftcircuit = QuantumCircuit(5)
2 init_state = np.kron(np.kron(np.kron(np.kron([1,1],[1,1]),[1,1]),[1,1]),[1,-1])
3 init_state_normal = init_state/np.linalg.norm(init_state)
4 qftcircuit.initialize(init_state_normal,[0,1,2,3,4])
5 qftcircuit.append(qft5,[0,1,2,3,4])
6 psi=Statevector(qftcircuit)
7 psi.draw(output='latex')

```

Salida: $|10000\rangle$

b) $|\phi_2\rangle = \frac{1}{2^{5/2}}(|0\rangle + |1\rangle)(|0\rangle + |1\rangle)(|0\rangle + |1\rangle)(|0\rangle - |1\rangle)(|0\rangle + |1\rangle)$

```

1 qftcircuit=QuantumCircuit(5)
2 init_state=np.kron(np.kron(np.kron(np.kron([1,1],[1,1]),[1,1]),[1,-1]),[1,1])
3 init_state_normal = init_state/np.linalg.norm(init_state)
4 qftcircuit.initialize(init_state_normal,[0,1,2,3,4])
5 qftcircuit.append(qft5,[0,1,2,3,4])
6 psi=Statevector(qftcircuit)
7 psi.draw(output='latex')

```

Salida: $\left(\frac{1}{2} + \frac{i}{2}\right) |01000\rangle + \left(\frac{1}{2} - \frac{i}{2}\right) |11000\rangle$

c) $|\phi_3\rangle = \frac{1}{2^{5/2}}(|0\rangle + |1\rangle)(|0\rangle + |1\rangle)(|0\rangle - |1\rangle)(|0\rangle + |1\rangle)(|0\rangle + |1\rangle)$

```

1 qftcircuit=QuantumCircuit(5)
2 init_state=np.kron(np.kron(np.kron(np.kron([1,1],[1,1]),[1,-1]),[1,1]),[1,1])
3 init_state_normal = init_state/np.linalg.norm(init_state)
4 qftcircuit.initialize(init_state_normal,[0,1,2,3,4])

```



```

5 qftcircuit.append(qft5,[0,1,2,3,4])
6 psi=Statevector(qftcircuit)
7 psi.draw(output='latex')

```

Salida:

$$(0,25 + 0,6035533906i)|00100\rangle + (0,25 + 0,1035533906i)|01100\rangle \\ + (0,25 - 0,1035533906i)|10100\rangle + (0,25 - 0,6035533906i)|11100\rangle$$

d) $|\phi_4\rangle = \frac{1}{2^{5/2}}(|0\rangle + |1\rangle)(|0\rangle - |1\rangle)(|0\rangle + |1\rangle)(|0\rangle + |1\rangle)(|0\rangle + |1\rangle)$

```

1 qftcircuit=QuantumCircuit(5)
2 init_state=np.kron(np.kron(np.kron(np.kron([1,1],[1,-1]),[1,1]),[1,1]),[1,1])
3 init_state_normal = init_state/np.linalg.norm(init_state)
4 qftcircuit.initialize(init_state_normal,[0,1,2,3,4])
5 qftcircuit.append(qft5,[0,1,2,3,4])
6 psi=Statevector(qftcircuit)
7 psi.draw(output='latex')

```

Salida:

$$(0,125 + 0,6284174365i)|00010\rangle + (0,125 + 0,1870757203i)|00110\rangle \\ + (0,125 + 0,0835223297i)|01010\rangle + (0,125 + 0,0248640459i)|01110\rangle \\ + (0,125 - 0,0248640459i)|10010\rangle + (0,125 - 0,0835223297i)|10110\rangle \\ + (0,125 - 0,1870757203i)|11010\rangle + (0,125 - 0,6284174365i)|11110\rangle$$

III. PRÁCTICA 3: EJECUCIÓN EN LOS ORDENADORES DE IBM

1. Generación de números aleatorios

1.1. Definir el mismo circuito de la Práctica 2

```
1 # How many random numbers will be produced in a single shot:
2 # Prepare the input circuit.
3 Nnumbers=32 # must be smaller than [127 qubits/3]=42
4 QRNG = QuantumCircuit(3*Nnumbers) # four qubits
5 for j in range(Nnumbers):
6     QRNG.h([(3*j+0),(3*j+1),(3*j+2)]) # apply Hadamard gate to each of the qubits
7     QRNG.measure_all() # measure all qubits
8 # QRNG.draw(output="mpl") # plot circuit: don't do it with many qubits
```

Se avisa en la última línea de que no se dibuje el circuito con muchos qubits, en este caso, 32 es un número demasiado grande como para representarlo.

Se obtiene el siguiente histograma:

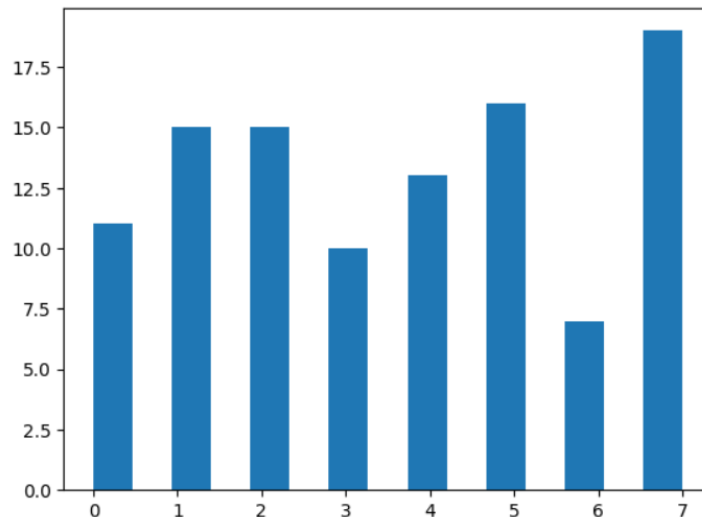


Figura 43: Histograma 32 qubits.

1.2. Comentario de los resultados

Se dibuja un histograma para ilustrar lo que sucede con cada uno de los qubits. En él, la frecuencia relativa de cada uno de los 7 números es muy parecida.

2. Algoritmo de DEUTSCH

Dado que en esta parte de la Práctica 3 se pide en casi todos los apartados completar los códigos, se han pegado todos los utilizados a lo largo de esta.

2.1. Demostrar la unitariedad del siguiente operador

Recordando las bases utilizadas para demostrar los casos del punto 2.1, podemos comprobar: $\langle xy|x'y' \rangle = \langle x(y \otimes f(x))|x'(y' \otimes f(x')) \rangle$. Y tenemos:

$$\begin{aligned} \langle xy|x'y' \rangle &= \langle x|x' \rangle \langle y|y' \rangle \\ \langle x(y \oplus f(x))|x'(y' \oplus f(x')) \rangle &= \langle x|x' \rangle \langle y \oplus f(x)|y' \oplus f(x') \rangle \end{aligned} \quad (4)$$

- Si $(x \neq x')$: $\langle x|x' \rangle = 0$ y los dos productos escalares de arriba son 0.
- Si $(x = x')$: $\langle x|x' \rangle = 1$ y se puede simplificar a: $f(x) = f(x')$.
 $\langle y \oplus f(x)|y' \oplus f(x') \rangle = \langle y \oplus f(x)|y' \oplus f(x) \rangle$. Y tenemos dos subespacios para y :
 - Si $(y \neq y')$: Entonces $\langle a|a' \rangle = 0$ y $\langle a \oplus b|a' \oplus b \rangle = 0$
 - Si $(y = y')$: Entonces $\langle a|a \rangle = 1$ y $\langle a \oplus b|a \oplus b \rangle = 1$

Se concluye entonces que el producto escalar se preserva y que U es unitario: $U^\dagger U = \mathbb{I}$.

2.2. Completar los códigos

```

1 #Classical check Oracle_f: f(0)
2 Check_f0 = QuantumCircuit(2,1)
3 #Remember that by default the two qubits are initialized to zero
4 Check_f0.unitary(Oracle_f, [0, 1],label="Oracle_f") # Apply Oracle_f
5 # Measure qubit 1
6 Check_f0.measure(1,0)
7 Check_f0.draw(output='mpl')

```

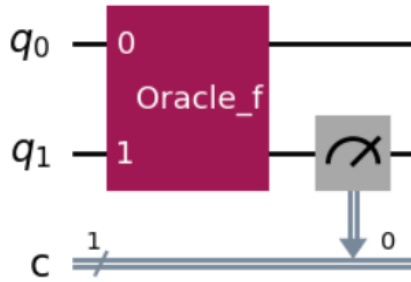


Figura 44: Circuito para $f(0)$ del oráculo sobre $|0\rangle$.

```

1 ##Classical check Oracle_f: f(1)
2 Check_f1 = QuantumCircuit(2,1)
3 #We initialize the first qubit to 1:
4 Check_f1.initialize([0,1],0)
5 Check_f1.unitary(Oracle_f, [0, 1],label="Oracle_f") # Apply Oracle_f
6 # Measure qubit 1
7 Check_f1.measure(1,0)
8 Check_f1.draw(output='mpl')

```

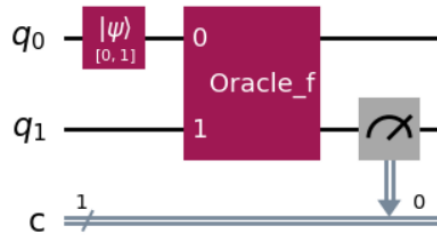


Figura 45: Circuito para $f(1)$ del oráculo sobre $|1\rangle$.

Obtener los valores de $f(0)$ y $f(1)$:

```

1 # One shot for each circuit
2 shots_C=1
3 # Sampler with aer simulator as backend
4 sampler_S = Sampler(mode=sim);
5 # The sampler evaluates probabilities of each input circuit
6 jobs_f = sampler_S.run([Check_f0,Check_f1],shots=shots_C).result();
7 result_f_0 = jobs_f[0].data.c.get_counts() # result of Check_f0 circuit
8 result_f_1 = jobs_f[1].data.c.get_counts() # result of Check_f1 circuit

```

Para estar completo, deberíamos mostrar los resultados en pantalla. Esto se ve en el siguiente punto.

2.3. Medir en f

```

1 #Print the measurement results
2 print(f"f(0)={max(result_f_0)}")
3 print(f"f(1)={max(result_f_1)}")

```

Con una salida de $f(0)=1$ y $f(1)=0$.

La explicación es que si se parte del estado $|00\rangle$ y se le aplica el operador definido $Oracle_f$:

$$\begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} \quad (5)$$

El resultado es el estado $|01\rangle$

Haciendo lo mismo pero con estado inicial $|10\rangle$, se obtiene el estado $|10\rangle$.

Esto coincide con la salida del código ya que si se mete en el primer qubit un 1 sale un 0 en el segundo y al revés.

2.4. Medir en g

```

1 #Classical check Oracle_g: g(0)
2 Check_g0 = QuantumCircuit(2,1)
3 Check_g0.unitary(Oracle_g, [0, 1],label="Oracle_g") # Apply Oracle_g
4 # Measure qubit 1
5 Check_g0.measure(1,0)
6 Check_g0.draw(output='mpl')

```

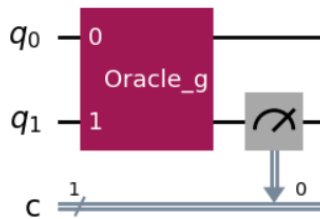


Figura 46: Circuito para $g(0)$ del oráculo sobre $|0\rangle$.

```

1 #Classical check Oracle_g: g(0)
2 Check_g0 = QuantumCircuit(2,1)
3 Check_g0.unitary(Oracle_g, [0, 1],label="Oracle_g") # Apply Oracle_g
4 # Measure qubit 1
5 Check_g0.measure(1,0)
6 Check_g0.draw(output='mpl')

```

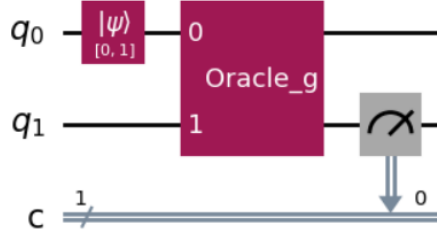


Figura 47: Circuito para $g(1)$ del oráculo sobre $|1\rangle$.

```

1 jobs_g = sampler_S.run([Check_g0, Check_g1], shots=shots_S);
2 result_g_0 = jobs_g.result()[0].data.c.get_counts()
3 result_g_1 = jobs_g.result()[1].data.c.get_counts()

1 #Print the measurement results
2 print(f"g(0)={max(result_g_0)}")
3 print(f"g(1)={max(result_g_1)}")

```

Con una salida de $g(0)=1$ y $g(1)=1$.

Tomando de nuevo $|00\rangle$ como estado inicial:

$$\begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} \quad (6)$$

Que da el estado $|01\rangle$, es decir, teniendo 0 para el primer qubit, en el segundo se obtiene 1.

Para un estado inicial $|10\rangle$, la salida es $|11\rangle$. Lo cual coincide con la salida del código ya que si al primer qubit se le da el valor 1, la salida para el segundo es 1.

2.5. Demostración si $f(0) \neq f(1)$

Si $f(0) \neq f(1)$, como estamos en congruencias módulo 2, entonces $1 \oplus f(0) = f(1)$ y $1 \oplus f(1) = f(0)$.

Para demostrarlo, vamos a ver cómo actúa el operador *Oracle* sobre el estado inicial $|+\rangle \otimes |-\rangle$. Sabiendo que $|x\rangle|y\rangle \rightarrow |x\rangle|y \oplus f(x)\rangle$ y que $1 \oplus f(0) = f(1)$ y $1 \oplus f(1) = f(0)$:

$$\begin{aligned} & \frac{1}{2} [0f(0) - 01 \oplus f(0) + 1f(1) - 11 \oplus f(1)] = \\ &= \frac{1}{2} [0f(0) - 0f(1) + 1f(1) - 1f(0)] = \\ &= \frac{1}{2} [0(f(0) - f(1)) + 1(f(1) - f(0))] = \\ &= \frac{1}{2} (0 - 1)(f(0) - f(1)) = \\ &= \frac{0 - 1}{\sqrt{2}} \frac{f(0) - f(1)}{\sqrt{2}} = \\ &= - \left[\frac{f(0) - f(1)}{\sqrt{2}} \right] \end{aligned} \quad (7)$$

2.6. Definir un circuito para implementar el algoritmo de Deutsch con f y g

```

1 # Deutsch's for f
2 deutsch_f = QuantumCircuit(2,1)
3 deutsch_f.x(1) # Apply X gate to qubit 0
4 deutsch_f.h([0, 1]) # Apply Hadamard gate to both qubits
5 deutsch_f.unitary(Oracle_f, [0, 1],label="Oracle_f") # Apply Oracle_f
6 deutsch_f.h(0) # Apply Hadamard gate to qubit 0
7 # Measure qubit 0
8 deutsch_f.measure(0,0)
9 deutsch_f.draw(output='mpl')

```

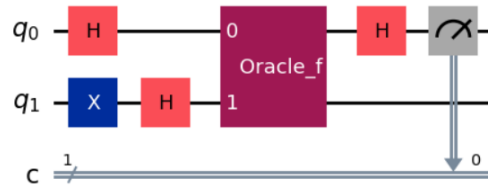


Figura 48: Circuito Deutsch con el oráculo f .

```

1 # Deutsch's for g
2 deutsch_g = QuantumCircuit(2,1)
3 deutsch_g.x(1) # Apply X gate to qubit 0
4 deutsch_g.h([0, 1]) # Apply Hadamard gate to both qubits
5 deutsch_g.unitary(Oracle_g, [0, 1],label="Oracle_g") # Apply Oracle_g
6 deutsch_g.h(0) # Apply Hadamard gate to qubit 0
7 # Measure qubit 0
8 deutsch_g.measure(0,0)
9 deutsch_g.draw(output='mpl')

```

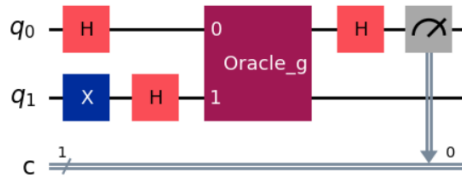


Figura 49: Circuito Deutsch con el oráculo g .

```

1 # Execute the circuit
2 jobs_DeutschS=sampler_S.run([deutsch_f,deutsch_g],shots=shots_S).result() # one
   entry for each function
3 result_DeutschS_f = jobs_DeutschS[0].data.c.get_counts()
4 result_DeutschS_g = jobs_DeutschS[1].data.c.get_counts()

```

```

1 # Print the measurement results for f, if 0 the fuction is constant
2 if max(result_DeutschS_f)=='0':
3     print("f_is_constant")
4 else:
5     print("f_is_not_constant")

```

Con salida "f is not constant".

```

1 # Print the measurement results for g, if 0 the fuction is constant
2 if max(result_DeutschS_g)=='0':
3     print("g_is_constant")
4 else:
5     print("g_is_not_constant")

```

Con salida "g is constant".

2.7. Comparación con el resultado de antes

```
1 shots_Q=3
2 sampler_Q = Sampler(mode=backend_Q) #to obtain the sampling of the measurement
3 # Execute the circuit
4 job_Q=sampler_Q.run([transpile(deutsch_f,backend_Q),transpile(deutsch_g,
    backend_Q)],shots=shots_Q)
```

En este caso, las salidas también son que f es no constante y que g es constante, como en el caso anterior, por lo que coinciden.

3. Desigualdades de Bell CHSH

3.1. Definir el circuito

```
1 # Prepare the input circuit:
2 chsh_circuit = QuantumCircuit(2)
3 chsh_circuit.h(0)
4 chsh_circuit.cx(0,1)
5 chsh_circuit.z(0)
6 chsh_circuit.x(0)
7 chsh_circuit.draw(output='mpl')
8 psi = Statevector(chsh_circuit)
9 psi.draw(output='latex')
```

Sale: $\frac{\sqrt{2}}{2}|01\rangle - \frac{\sqrt{2}}{2}|10\rangle$

3.2. Definir los cuatro productos de observables

```
1 # Define pairs of observables for maximum violation of the CHSH innequality
2 A1B1=SparsePauliOp.from_list([('XX', -1/sqrt(2)), ('XY', -1/sqrt(2))])
3 A1B2=SparsePauliOp.from_list([('XX', -1/sqrt(2)), ('XY', 1/sqrt(2))])
4 A2B1=SparsePauliOp.from_list([('YX', -1/sqrt(2)), ('YY', -1/sqrt(2))])
5 A2B2=SparsePauliOp.from_list([('YX', -1/sqrt(2)), ('YY', 1/sqrt(2))])
6 Obs=[A1B1,A1B2,A2B1,A2B2] #The four correlators are packaged in one list of
    observables
```

Los valores esperados para los 4 productos [A1B1,A1B2,A2B1,A2B2] han sido [0.7122858 0.7019278 0.6960582 -0.7181553], y las desviaciones típicas [0.011, 0.011, 0.011, 0.011047195]

3.3. Calcular el valor de la desigualdad CHSH

```
1 CHSH_mean = result_S.data.evs[0] + result_S.data.evs[1] + result_S.data.evs[2]
    - result_S.data.evs[3]
2 CHSH_uncertainty=sqrt(Standard_errors_S[0]**2 + Standard_errors_S[1]**2 +
    Standard_errors_S[2]**2+Standard_errors_S[3]**2)
3 print(f"The simulated result is {CHSH_mean} + {CHSH_uncertainty}: \ndoes it
    exceed 2 with sufficient statistical certainty?")
```

Con salida: "The simulated result is 2.82842712474619 + 0.022023635425788016: does it exceed 2 with sufficient statistical certainty?"

Este valor simulado es mayor que 2, por lo que viola la desigualdad de Bell incluso teniendo en cuenta la incertidumbre.

3.4. Valor experimental para CHSH

Para el cálculo en el caso cuántico se utiliza el mismo código que para el anterior.

Los valores esperados han sido [0.6309711 0.7444791 0.7018209 -0.5912804] y las desviaciones típicas [0.011, 0.011, 0.011, 0.011047195].

Así como la salida al calcular la desigualdad de Bell que ha sido:

"The simulated result is $2.82842712474619 + 0.022023635425788016i$: does it exceed 2 with sufficient statistical certainty?"

Este resultado también viola la desigualdad de Bell.

IV. ANEXOS

En esta sección se facilita el acceso al material utilizado en la práctica.

- **Guion de la Práctica:** El documento donde se explican todos los ejercicios a realizar para la Práctica 1 es el siguiente. [Acceder al Guión.](#)
- **Cuadernos de Jupyter:** Todos los cuadernos de Jupyter proporcionados por el profesor con la extensión de las líneas de código pertinentes para poder realizar los ejercicios se encuentran en el siguiente link. [Acceder a los Notebooks.](#)